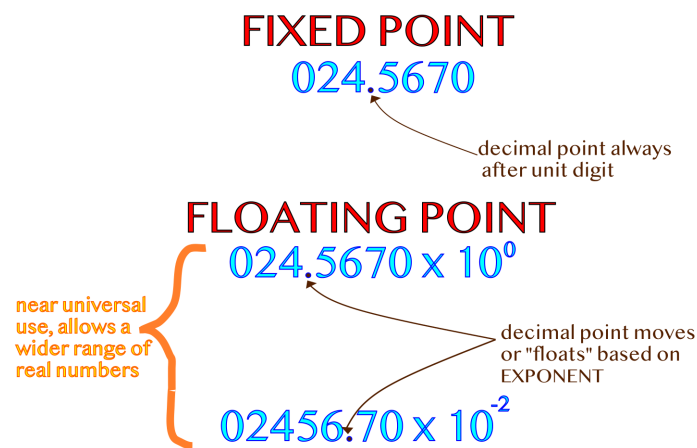


Getting Right to the Point: Approximations of Floating Point Numbers

Jeremy Ku-Benjet

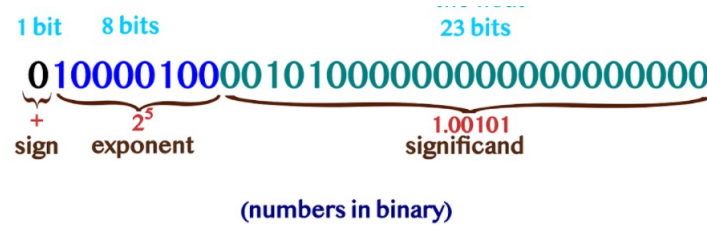
Long ago, 16-bit microprocessors were state of the art. Each computer came with its own way of representing real numbers. Floating point numbers, numbers where the decimal point can move, were almost universally preferred.



However, different implementations of floating point numbers or “floats” each came with their own quirks. Suffice to say, when subtracting two distinct numbers results in zero in some, but not all, implementations of floating point arithmetic, there is a problem. To combat the chaos, there needed to be standardization, and so efforts to unify floats under the name of the IEEE (Institute of Electrical and Electronics Engineers) began in the late 1970s. Meetings included representatives from Motorola, Intel, National Semiconductors, and many of the other major microprocessor manufacturers. It was a unified effort to agree on a good standard for floating point numbers. Of course, there was still strong debate over the details of the standard, delaying its creation for several years. Slowly, the fundamentals of the new standard were agreed upon, and by 1984 it was implemented nearly universally by microprocessor manufacturers. The following year it was officially published as IEEE 754 and since has been used, not just for hardware, but for representing numbers in software as well. In the most common representation, a floating point number is structured the same

as a number in scientific notation. There is a sign, some significant digits, called the significand (or

mantissa), and an exponent. Computers find it convenient to use base 2, so it will be used for the remainder of this article.



According to IEEE 754, a single precision floating point number is represented by 32 bits. The first bit is the sign bit, 1 if the number is negative, else 0. Following is the exponent, an 8-bit positive integer, 127 more than the exponent of the 2. Storing the exponent with this “bias” allows for negative exponents and therefore numbers between 1 and -1. The final 23 bits are the significand (or mantissa). They describe the details of a number. So there are no leading zeros, which could more efficiently be stored in the exponent, the significand is “normalized.” If the significant digits of the number are 1.00101 then the significand is 00101. The 1 in the significant digits is implicitly stored. As the number is normalized, the 1 is always present so storing it in the significand is needless.

$$\begin{array}{c}
 \text{stored implicitly} \\
 \text{1.011} \times 2^{-2} \\
 \text{NOT} \\
 \text{0.1011} \times 2^{-1}
 \end{array}$$

There are some special floats which don’t follow the normal format, such as zero, but listing them here would be superfluous.

Integers can be converted into floats. A program can easily figure out the sign, exponent, and significand; however, almost any modern computer will have an instruction to convert an integer into a float faster than could be written otherwise. To find the exponent of the float, the computer computes $2n$, which can be useful to know. Therefore, by taking the second through ninth bits of a float, converting an integer to a float can be used to quickly get the binary logarithm of a number. The limitation of this approximation is precision. The method to get more precision is to take n to a power:

$$\frac{\log_2 n^m}{m} = \frac{m \log_2 n}{m} = \log_2 n$$

A larger input to the $\log_2$ means the integer portion of the output will be larger and the final output can be made more precise:

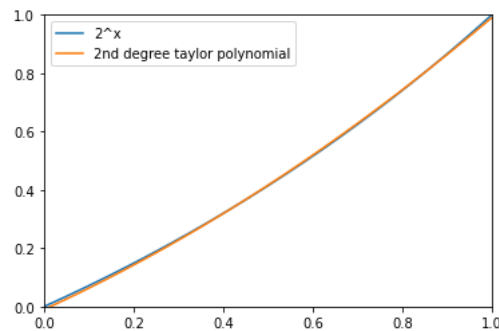
$$\lfloor \log_2 10 \rfloor = 3 \text{ and } \frac{\lfloor \log_2 10^{10} \rfloor}{100} = 3.3$$

However, the powers of the input grow exponentially and can quickly become too large to be easily handled. Getting another decimal digit of precision requires handling 10^{100} , a number too large to be stored in a standard 4-byte or 8-byte integer.

A float does get around having to take the binary logarithm; it just passes the operation off to a faster part of the program. Converting an integer to a float and reading the exponent to approximate a logarithm suggests writing to the exponent and interpreting the data as an integer, in a way, the inverse of how a logarithm was approximated. This method could also be used to approximate an exponential. This becomes clear, as the structure of 2^x , when broken into a fractional and integral part, mirrors a float's representation. Breaking up x into $[x]$ and x , the integer part and the fractional part of x respectively, $2^{[x]+x} = 2^{[x]} \cdot 2^x$. $2^{[x]}$ is an integer, with $[x]127$ less than the exponent of the float (due to the bias), and $0 \leq 2^x < 2$ is a fraction, resembling the significand.

$$2^x = \underbrace{(1 + 2^{\{x\}} - 1)}_{\text{approximated}} \times \underbrace{2^{([x]+127)-127}}_{\text{computed when reinterpreting the float}}$$

The only unknown information about the float representing 2^x is the significand, $2^x - 1$. The minus 1 is due to the implicit 1 stored in a float. Therefore, the significand must be approximated. However, approximating $2^x - 1$ is much easier than the original problem of approximating 2^x as $0 \leq x < 1$. The range $\{x\}$ is relatively small, so using a Taylor polynomial to accurately approximate all $2^x - 1$ is possible.



Combining the calculated exponent and approximated significand into a single float creates an efficient method to take 2 to any power.

Writing $n = (1 + s)2^e$ where n is a number, s is the significand, and e is the exponent, and $N = (1 + s)2^{23} + e * 2^{23}$, where N is interpreting the float representation of n as an integer, allows for alternate approximations of exponents and logarithms. Note the 2^{23} in the integer interpretation of the float comes from the significand being 23 bits long.

Taking $\log(n) = \log(1 + s)2^e$ and using N to solve for e allows for $\log(n)$ to be rewritten in a more convenient form such that only $\log(1 + s) - s$ must be approximated. As s can only range from zero to one, this approximation is possible with accuracy.

References

- IEEE Computer Society (2008). IEEE Standard For Floating Point Arithmetic. Retrieved June 18, 2021. <https://irem.univ-reunion.fr/IMG/pdf/ieee-754-2008.pdf>
- Paul, M. <http://www.machinedlearnings.com/2011/06/fast-approximate-logarithm-exponential.html>
- Sree, K (2006) Know your FPU: Fixing Floating Point Fast Retrieved June 18 2021. <http://stereopsis.com/sree/fpu2006.html>
- William, K (1998). An Interview With The Old Man of Floating Point. Retrieved June 18, 2021. <https://people.eecs.berkeley.edu/~wkahan/ieee754status/754story.html>.